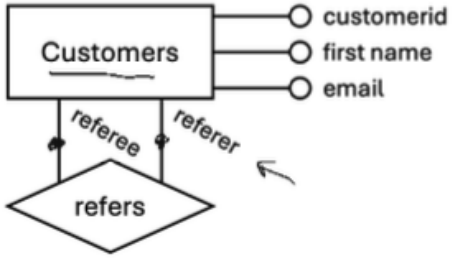
CS4221
AY25/26 SEM 2
github/Gavino3o

Entity Relationship Diagrams

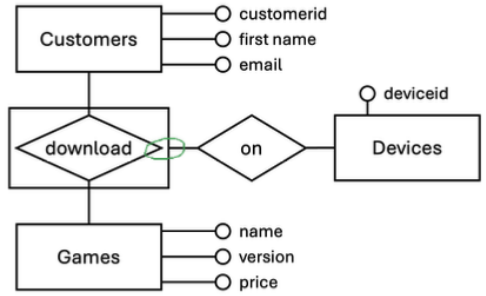
- Entity-relationship model is value-oriented. Values are atomic data types (INTEGER, TEXT, etc).
- Entity sets: rectangles, with name in plural noun form.
- Attributes: circles, with name in singular noun form.
- Derived attributes: dashed line connected to a circle. (May be derived from other attributes, or from other entity sets.)
- Relationship sets: diamonds, with name in verb form. Connected to entity sets with lines.
- **No duplicates** for entity and relationship sets
- Relationships can have attributes. However, **relationships are not distinguished by their attributes**, but by the entities they connect. **No black circle!** (e.g cannot have customer downloading same game multiple times on different devices)
- **n-ary relationships:** relationship sets that connect more than 2 entity sets. All entities must be participating; no entity can be missing.

Diagram



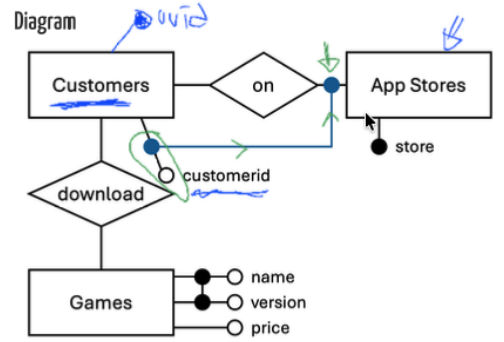
- **Attribute Promotion**: Converting an attribute of a relationship set into an entity set. This is done when the attribute has multiple values for the same relationship (e.g. a customer can download the same game on multiple devices, so we promote the "device" attribute into an entity set).
- **Aggregate**: A relationship set can be treated as an entity set when it participates in another relationship set. This is done by enclosing the relationship set in a rectangle, and connecting it to the new relationship set with a line.

Diagram



- **Identifying Attributes**: One or more UNIQUE NOT NULL attributes that can be used to identify an entity. Black Circle.
- Entity set can have multiple candidate keys, but only one primary key.
- **Partial Identification**: When an entity cannot be identified by its own attributes alone, but can be identified by a combination of its own attributes and the identifying attributes of another entity set. Connect the partial identifier (attribute) to the participation of dominant entity.
- Can have multiple dominant entity sets. Also can have other (non-partial) key.

Diagram

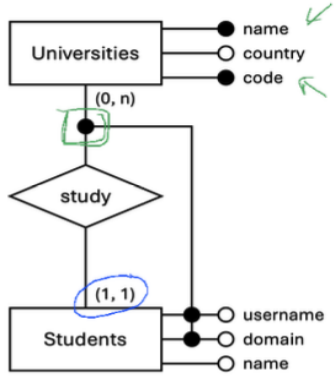


- **Participation Constraints**: Between entity and relationship set. Restricted by (minimum, maximum) cardinality. **Default: (0, n)**

Common Names

- `(1, _)` characterizes a ***mandatory*** participation.
- `(0, _)` characterizes an ***optional*** participation.
- `(_, 1)` may characterizes a ***one-to-one*** relationship
  if `(_, 1)` for all entities involved
- `(_, 1)` may characterizes a ***one-to-many*** relationship.
  if `(, 1)` for one but `(, n)` or `(_, x)` for $x > 1$ for other
- `(_, n)` characterizes a ***many-to-many*** relationship.
  if `(_, n)` for all

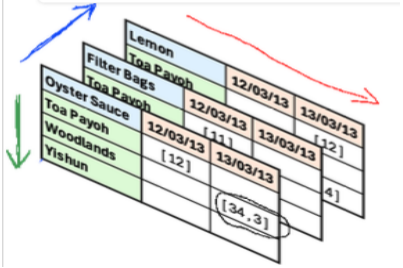
- Weak entities can only be defined for a (1, 1) constraint.



- Translating ER to Relational Schema:
  - **Rule 1**: Map ER attributes to attributes of relations with meaningful types. (VARCHAR, INTEGER, etc)
  - **Rule 2**: Map each entity set to a relation. Candidate keys are mapped to primary keys and UNIQUE NOT NULL.
  - Composite keys can be specified using table constraints. (PRIMARY KEY (attr1, attr2))
  - **Rule 3**: Map each relationship set to a relation. The relation will have the primary keys of the participating entity sets as foreign keys, and any attributes of the relationship set as attributes of the relation. The primary key of the relation will be the combination of the primary keys of the participating entity sets. (PRIMARY KEY (entity1id, entity2id))
  - Treat aggregates as a relationship set. But its keys can be **referenced** by other relationship sets.
  - **Exception 1**: (0, 1) One-to-many relationship sets can be enforced by removing the keys of the "many" side from the primary key.
  - **Exception 2**: (1, 1) entities in a relationship. Need to enforce minimum cardinality. We merge the entity and the relationship set into a single relation, with the primary key of the entity set as the primary key of the relation.
  - **Exception 3**: If partial identification is used, we can merge the weak entity set with the relationship set into a single relation, with the primary key of the dominant entity set and the partial identifier as the primary key of the relation.

Data Warehousing

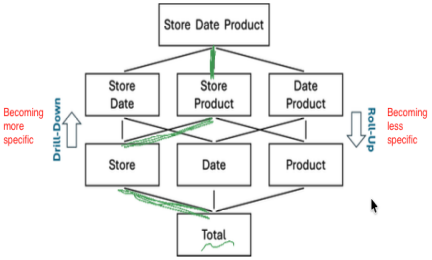
- **OLTP:** Online Transaction Processing. Focuses on handling a large number of transactions, with a small amount of data per transaction, data up-to-date and consistent, concurrency. (e.g. banking, e-commerce)
- **OLAP:** Online Analytical Processing. Focuses on handling a small number of transactions, with a large amount of data per transaction, static snapshots, query require lots of resources. (e.g. data warehousing, business intelligence)
- Kimball's principles for Data Warehousing: Consistency, Accessibility, Security, Adaptability, Acceptance, Decision Support.
- **Dimension Modelling**: A technique for designing data warehouses. It consists of a fact table and dimension tables. The fact table contains the measures (e.g. sales amount, quantity) and foreign keys to the dimension tables. The dimension tables contain the attributes of the dimensions (e.g. product name, customer name).
- Each column becomes a dimension.



- **Star Schema**: A fact table with multiple dimension tables. The dimension tables are not normalized. (e.g. product dimension table has product name, product category, etc)
- **Fact Table:** Records, for each business process, the measures (aka facts). It also records, for each dimension, the foreign/surrogate key to the dimension table. (e.g. sales fact table has sales amount, quantity, and foreign keys to product, customer, time dimension tables)
- Finest available grainularity. Surrogate key is often integers, which are more efficient to join on than natural keys (e.g. product name).
- **Dimension Table:** Records the attributes of the dimensions. It also has a surrogate key as the primary key. (e.g. product dimension table has product, sku, name, category, etc)
- Dimension tables can have specialised rows to represent "unknown" or "not applicable" values. Better than **NULL** values.
- Dimension tables are often denormalised, with **redundant data** and also **precomputed values**. Improve query performance by reducing the number of joins.
- Hierarchy of dimensions: A dimension can have a hierarchy of attributes. (e.g. time dimension has year, month, day)
- **Snowflake Schema**: A fact table with multiple dimension tables, where the dimension tables are normalized. Avoids data redundancy, but requires more joins.
- **Slice**: Select a layer of the cube by fixing a dimension. WHERE clause.
- **Dice**: Select a subcube by fixing multiple dimensions. WHERE clause.
- **Window functions**: Perform calculations across a set of rows that are related to the current row. (e.g. RANK(), ROW_NUMBER(), etc)
- **Roll-up**: Aggregate the data by climbing up the hierarchy of a dimension. GROUP BY ROLLUP(B, C, A) equivalent to ordered union of GROUP BY (B, C, A), GROUP BY (B, C), GROUP BY (B), and GROUP BY ().

category	area	avgvolume	
Household Items	YISHUN	5178.07	
Groceries	YISHUN	3787.16	
null	YISHUN	8965.23	"subtotal"
:	:	:	
null	*null*	25168.51	"grand total"
Toiletries	*null*	6203.28	"subtotal"
:	:	:	

- **Drill-down**: Aggregate the data by climbing down the hierarchy of a dimension. Normal GROUP BY clause.



- **CUBE**: subclause of GROUP BY that generates all possible combinations of the dimensions. (e.g. GROUP BY CUBE (A,B C) ordered . union of GROUP BY of all combinations of A, B, C). Null values are included.

category	area	avgvolume	
Household Items	YISHUN	5178.07	
Groceries	YISHUN	3787.16	
null	YISHUN	8965.23	"subtotal"
:	:	:	
null	*null*	25168.51	"grand total"
Toiletries	*null*	6203.28	"subtotal"
:	:	:	

- **Bus Architecture**: A data warehouse architecture where the fact tables and dimension tables are shared across multiple data marts.
- Data Integration with ETL → dimension modelling for OLAP → data optimization → OLAP tools.

Relational Algebra

- **Selection** (σ): Selects rows from a relation that satisfy a given predicate. (e.g. $\sigma_{[age>30]}(Employees)$). WHERE clause.
- **Projection** (π): Selects columns from a relation. (e.g. $\pi_{[name,age]}(Employees)$). SELECT clause.
- **Renaming** (ρ): Renames a relation or its attributes. (e.g. $\rho(R_1, R_2)$ renames the relation R_1 to R_2). AS clause.
- Example: $\pi[r.attr](\sigma[c](\rho(rel, r)))$
- **Set Operations**: Union ($\cup$), Intersection ($\cap$), Difference ($-$). (e.g. $R \cup S$, $R \cap S$, $R - S$). Requires the relations to be union-compatible (same number of attributes, and corresponding attributes have the same domain).
- **Cross Product** ($\times$): Combines each row of one relation with each row of another relation. (e.g. $R \times S$). Cartesian product. Nested loops join.
- Conjunction ($\wedge$) and Disjunction ($\vee$)
- **Sigma Cascade**: A selection with a conjunction of predicates can be expressed as a cascade of selections. (e.g. $\sigma_{[c_1 \wedge c_2]}(R)$ equivalent to $\sigma_{[c_1]}(\sigma_{[c_2]}(R))$). Evaluation can be more efficient if the **inner selection produces fewer rows**.
- **Selection Distributivity over Cross Product**: A selection with a predicate that only involves attributes from one relation can be distributed over the cross product. (e.g. $\sigma_{[c(R)]}(R \times S)$ equivalent to $\sigma_{[c(R)]}(R) \times S$). Evaluation can be more efficient if the selection is applied before the cross product, as it reduces the number of rows in the intermediate result.
- **Selection Distributivity over Union**: A selection with a predicate can be distributed over a union. (e.g. $\sigma_{[c]}(R \cup S)$ equivalent to $\sigma_{[c]}(R) \cup \sigma_{[c]}(S)$). Evaluation can be more efficient if the selection is applied before the union, as it reduces the number of rows in the intermediate result. Same applies to intersection and difference.
- **Pi Cascade**: A projection with a list of attributes can be expressed as a cascade of projections. (e.g. $\pi_{[a_1,a_2]}(R)$ equivalent to $\pi_{[a_1]}(\pi_{[a_1,a_2]}(R))$). Evaluation can be more efficient if the **inner projection produces fewer columns**.

Database Tuning

- PostgreSQL Architecture: SQL Parser and Rewriter → Query Planner and Optimizer → Executor → Memory and Storage Management.
- Planner/Optimizer: Generates query plans using catalogue and statistics, choose plan with least estimated cost.
- Cost is estimated based on the number of rows processed, the number of disk I/O operations, and the CPU time required for the operations.
- Execution Plan: DAG/Tree of operations.
- Types: Sequential Scan, Index Scan, Bitmap Index Scan, Bitmap Heap Scan, Nested Loop Join, Hash Join, Merge Join, Aggregate Operation etc.
- Total query time: query planning time + execution time, network latency, client overhead.
- **EXPLAIN (ANALYZE, VERBOSE)** command: shows the execution plan, execution time, row count for each step. (e.g. EXPLAIN ANALYZE SELECT * FROM Employees WHERE age > 30)
- Only supports: SELECT, INSERT, UPDATE, DELETE, EXECUTE, CREATE TABLE, DECLARE.
- In the query plan: (cost=startup cost..total cost rows=estimated rows width=estimated row width (in bytes)). Startup cost is the cost before the first row is returned, total cost is cost of executing the node (including its children).
- **pg_tables**: Catalog table that contains information about tables in the database. (e.g. SELECT * FROM pg_tables)
- **pg_indexes**: Catalog table that contains information about indexes in the database. (e.g. SELECT * FROM pg_indexes)
- **pg_stats/pg_statistics**: Catalog table that contains statistics about the distribution of values in the columns of the tables. Populated by the ANALYZE command (e.g. SELECT * FROM pg_stats)
- **pg_attributes**: Catalog table that contains information about the attributes (columns) of the tables. (e.g. SELECT * FROM pg_attributes)
- **Total time of a node** = product of actual time * loops
- **VACUUM**: Reclaims storage occupied by dead tuples. Also updates statistics for the query planner. (e.g. VACUUM ANALYZE Employees)
- **Index Types**: B-tree (default, supports equality and range queries), Hash (supports equality queries), GIN (supports full-text search), GiST (supports spatial data), BRIN (supports large tables with natural ordering).
- **Indexes** generally slow down write operations (INSERT, UPDATE, DELETE) because the index also needs to be updated. However, they can significantly speed up read operations (SELECT) by allowing the query planner to quickly locate the relevant rows.
- **PostgreSQL** automatically creates **B-tree unique indexes for primary key and unique constraints**. However, it does not automatically create indexes for foreign keys.
- **SYNTAX**: CREATE INDEX index_name ON table_name (column_name...);
- We can create unique indexes. CREATE UNIQUE INDEX...
- We can use WHERE to create partial indexes.
- **Clustering**: Reorganizes the physical storage of the table to match the order of an index. This can improve the performance of queries that use the index. (e.g. CLUSTER Employees USING idx_age)
- **A table can only be clustered on one index at a time.** Clustering needs to be **reapplied** after any write operation that modifies the clustered index.
- **Multi-column Indexes**: The order of the columns in the index matters. Index is used if condition involves **prefix** of the multi-column index (e.g. CREATE INDEX idx_age_name ON Employees (age, name))

Data Retrieval Operations

- **Sequential Scan**: Reads the entire table sequentially. Used when there is no index, or when the query planner estimates that it will be faster than using an index (e.g. when a large portion of the table is being scanned).
- **Sort**: Sorts the rows based on the specified columns. Used for ORDER BY, GROUP BY, DISTINCT, etc. (Quick Sort, Merge Sort, etc)
- **Grouping/Aggregation**: Groups the rows based on the specified columns and applies aggregate functions (e.g. SUM, COUNT, AVG) to each group. Used for GROUP BY, aggregate functions without GROUP BY, etc.
- **Unique**: Removes duplicate rows from the result set. Used for DISTINCT, etc. **Note:** Don't use DISTINCT for candidate key columns.
- **Index Only Scan**: Uses an index to retrieve the values of the indexed columns, without accessing the table. Used when index is a covering index.
- **Index Scan**: Uses an index to retrieve the values of the indexed columns, and then accesses the table to retrieve the values of the non-indexed columns. Used when there is an index on the columns used in the WHERE clause, but the index is not a covering index.
- **Bitmap Heap Scan**: Uses a bitmap index scan to retrieve the row locations of the matching rows, and then accesses the table to retrieve the values of the columns. Used when there is an index on the columns used in the WHERE clause, but the index is not a covering index, and the query planner estimates that it will be faster than using an index scan (e.g. when a large portion of the table is being scanned).
- **Bitmap Index Scan**: Uses a bitmap index to retrieve the row locations of the matching rows. Can be used for conjunctions or disjunctions of predicates on indexed columns.

PostgreSQL Cheatsheet

- **Syntax overview**: SQL is case-insensitive; statements end with ';'. Keywords: SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP, GRANT, REVOKE.
- **Common options**: 'EXPLAIN'/'EXPLAIN ANALYZE', 'SET' to change runtime parameters, 'BEGIN'/'COMMIT'/'ROLLBACK' for transactions, 'VACUUM', 'ANALYZE', 'CREATE INDEX CONCURRENTLY'.
- **Comparison qualifiers**: '> ALL', '< ALL', '>= ANY', 'SOME' synonyms; compare against every/any value from subquery. 'IS NULL'/'IS NOT NULL' for null tests.
- **Pattern matching**: 'LIKE' (case-sensitive), 'ILIKE' (case-insensitive); '%' = any string, '_' = single char. Use POSIX regex ' '/' *'/'! '/'! *'.
- **Existence**: 'EXISTS (subq)' returns true if subquery yields rows; 'NOT EXISTS' negates. Often correlated with outer query.
- **Membership**: 'expr IN (a,b,...)' or 'IN (subquery)'; 'NOT IN' negates but beware NULLs (use 'NOT EXISTS' if null-safe).
- **For-all logic**: express 'for every' via double 'NOT EXISTS' (or use 'ALL'/'>= ALL' where applicable). e.g. 'WHERE NOT EXISTS (SELECT 1 FROM t1 x WHERE NOT EXISTS (SELECT 1 FROM t2 y WHERE y.key=x.key AND ...))'. Alternates: anti-join pattern 'LEFT JOIN ... WHERE b.id IS NULL'.
- **Set operations**: UNION (remove duplicates), UNION ALL, INTERSECT, EXCEPT. All arms must have same column count and compatible types; order and column names come from first query.
- **COALESCE**: Returns the first non-null value in the list. (e.g. COALESCE(column1, column2, 'default')) (e.g. AVG(COALESCE(l.rd_date-l.bd_date, CURRENT_DATE - bd_date), 2) AS average)
- For MAX queries with potentially multiple rows with the same maximum value, we can use a subquery to find the maximum value, and then join it back to the original table to retrieve all rows with that maximum value. (e.g. SELECT name FROM Employees WHERE age = (SELECT MAX(age) FROM Employees))

```
CREATE TABLE downloads(
  customerid VARCHAR(16)
    REFERENCES customers(customerid),
  name       VARCHAR(32),
  version    CHAR(3),
  deviceid   VARCHAR(64) NOT NULL,
  PRIMARY KEY (customerid, name, version),
  FOREIGN KEY (name, version)
    REFERENCES games(name, version)
);
```